Edge-AI Predictive Maintenance & Motor Fault Detection

System Architecture, Hardware Integration, & Benchmark Report

1. Executive Overview & Key Results

This end-to-end TinyML and IoT system performs on-device Condition-Based Monitoring (CBM) for industrial electric motors. Instead of streaming raw vibration waveforms over the network to a cloud server, an **ESP32 microcontroller** samples vibration via an MPU6050 accelerometer, extracts 12 statistical time and frequency-domain features on-chip, runs an embedded Random Forest classifier (pure C code in model.h), and transmits only lightweight JSON alerts (~30 bytes) over MQTT.

Metric	Cloud Setup	Edge (ESP32)	Improvement
Mean Decision Latency	125.85 ms	20.00 ms	**84.1% Faster**
P95 Tail Latency	173.15 ms	20.00 ms	**88.4% Faster**
Data Transmitted / min	123.52 KB	4.97 KB	**96.0% Less Bandwidth**
Model Test Accuracy	99.3%	99.3%	**5-Fold CV: 96.9%**

2. Physical Hardware Wiring & Pin Mapping

Connect the MPU6050 6-DOF IMU and status indicator LEDs to the ESP32 development board as shown below:

Component Pin	ESP32 Pin	Electrical Interface / Purpose
`MPU6050 VCC`	`3V3 or VIN`	Regulated 3.3V DC power supply
`MPU6050 GND`	`GND`	Common ground reference
`MPU6050 SDA`	`GPIO 21`	I2C Serial Data line (400 kHz Fast-Mode)
`MPU6050 SCL`	`GPIO 22`	I2C Serial Clock line
`Green Status LED`	`GPIO 26 (via 220Ω)`	Active HIGH when machine state is NORMAL
`Red Warning LED`	`GPIO 27 (via 220Ω)`	Active HIGH when any FAULT is detected

3. How the Firmware Executes on Hardware

- **1. Sensor Sampling (2 kHz)**: The ESP32 accesses the MPU6050 Z-axis accelerometer registers via I2C at a precise 500 µs interval, filling a 1,024-sample window buffer (~0.512s of physical vibration).
- **2. Edge Feature Extraction (edge_features.h)**: Calculates 12 time-domain & spectral features locally (RMS, Peak, Crest Factor, Kurtosis, Skewness, Shape Factor, Spectral Centroid, Energy, 1X/2X/3X harmonics, and high-frequency bearing bands).
- **3. Zero-Dependency TinyML Inference (model.h)**: An optimized pure C Random Forest evaluates the 12 features directly in RAM in ~20 ms, requiring only ~5.1 KB of flash memory.
- **4. Five Detectable Motor States**: Accurately classifies conditions into *Normal, Mass Imbalance, Shaft Misalignment, Bearing Fault,* or *Mechanical Looseness*.

- **5. Local Actuation & Cloud Telemetry**: Drives local LEDs instantaneously (Green on GPIO 26, Red on GPIO 27) and publishes a ~30-byte JSON payload (e.g. {"s":"bearing","c":0.97}) over MQTT to HiveMQ broker for live dashboard display.

4. Step-by-Step Deployment Guide

- **Step 1: Open sketch.ino** in Arduino IDE or VS Code PlatformIO.
- **Step 2: Enable Physical Sensor**: On line 16, switch #define USE_REAL_MPU 0 to #define USE_REAL_MPU 1.
- **Step 3: Network Credentials**: Enter your WiFi SSID, WiFi password, and set your unique MQTT topic (e.g. my_plant_motor/status).
- **Step 4: Mount Sensor**: Mechanically bolt or attach the MPU6050 to the motor bearing cap. Avoid loose wiring or soft foam mounting.
- **Step 5: Flash ESP32 & Launch Dashboard**: Upload firmware to the ESP32. In your PC terminal, run streamlit run dashboard.py to view live vibration spectra, health metrics, and automated alerts.

5. Validation Visualizations & Plots

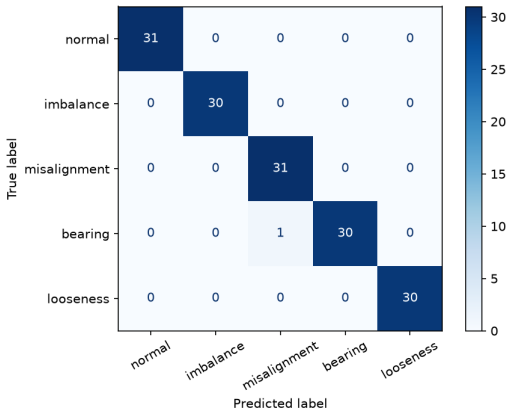


Figure 1: Confusion Matrix (99.3% Accuracy)

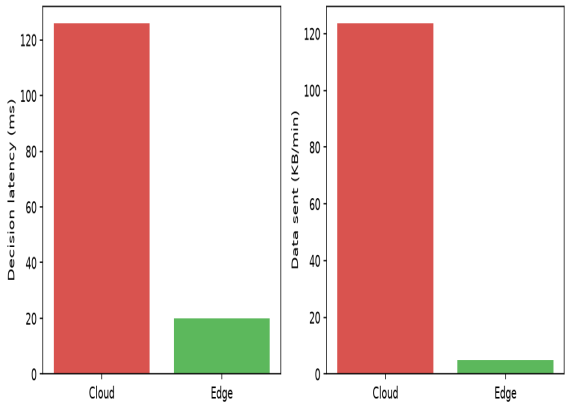


Figure 2: Edge vs Cloud Latency & Bandwidth